RENTAL MANAGEMENT SYSTEM

Backend API Development Report

Submitted By:

Tanishq Dhote
Prithviraj Kshirsagar
Sudesh Jambhulkar

Date: June 18, 2026

Table of Contents

1. Project Overview

The Rental Management System is a web service designed to facilitate tenancy leases, property listings, and secure user management. Built with FastAPI and backed by a relational PostgreSQL database, it provides robust APIs that allow landlords to list rental properties and tenants to search for and lease available accommodations.

Security is managed using industry-standard JSON Web Tokens (JWT) for authentication. Additionally, role-based authorization enforces access control, ensuring only authorized landlord accounts can modify property listings while tenant accounts are permitted to sign tenancy contracts.

2. Technology Stack & Tools Used

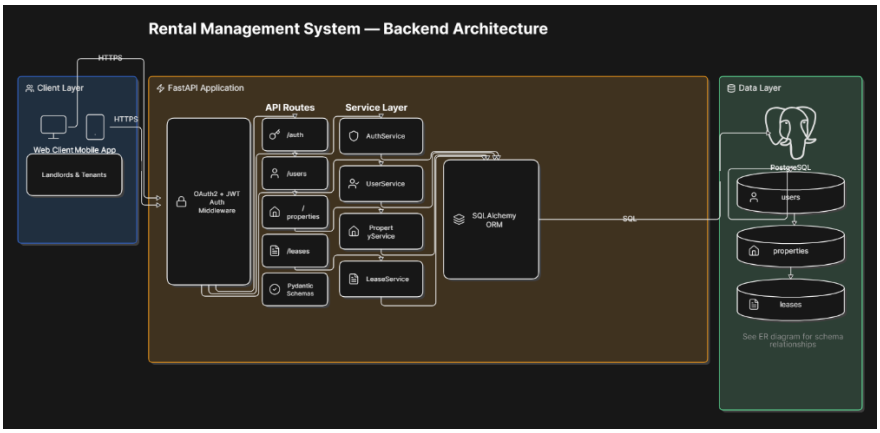
The following tools and frameworks were utilized to build and run the application:

Technology / Tool	Purpose & Role in Project
FastAPI	Python web framework used for designing the RESTful endpoints, validating requests, and generating automatic Swagger documentation.
PostgreSQL	Relational Database Management System (RDBMS) used for secure, relational, and persistent data storage.
SQLAlchemy	Object-Relational Mapping (ORM) library used to interact with the PostgreSQL database using Python classes (models) instead of raw SQL queries.
Uvicorn	Asynchronous Server Gateway Interface (ASGI) web server used to run the FastAPI application locally.
Pydantic	Data parsing and validation library used to define request and response payloads (schemas) with strict type checking.
python-jose & bcrypt	Libraries used to securely hash user passwords (using bcrypt) and generate/validate signed JSON Web Tokens (JWT).
Postman	API client used for manually calling, testing, and verifying the routes during development.

3. Database Schema Design (ER)

The relational database schema is structured into three main tables: **users**, **properties**, and **leases**. Below is the relational structure of the tables:

4. Architecture

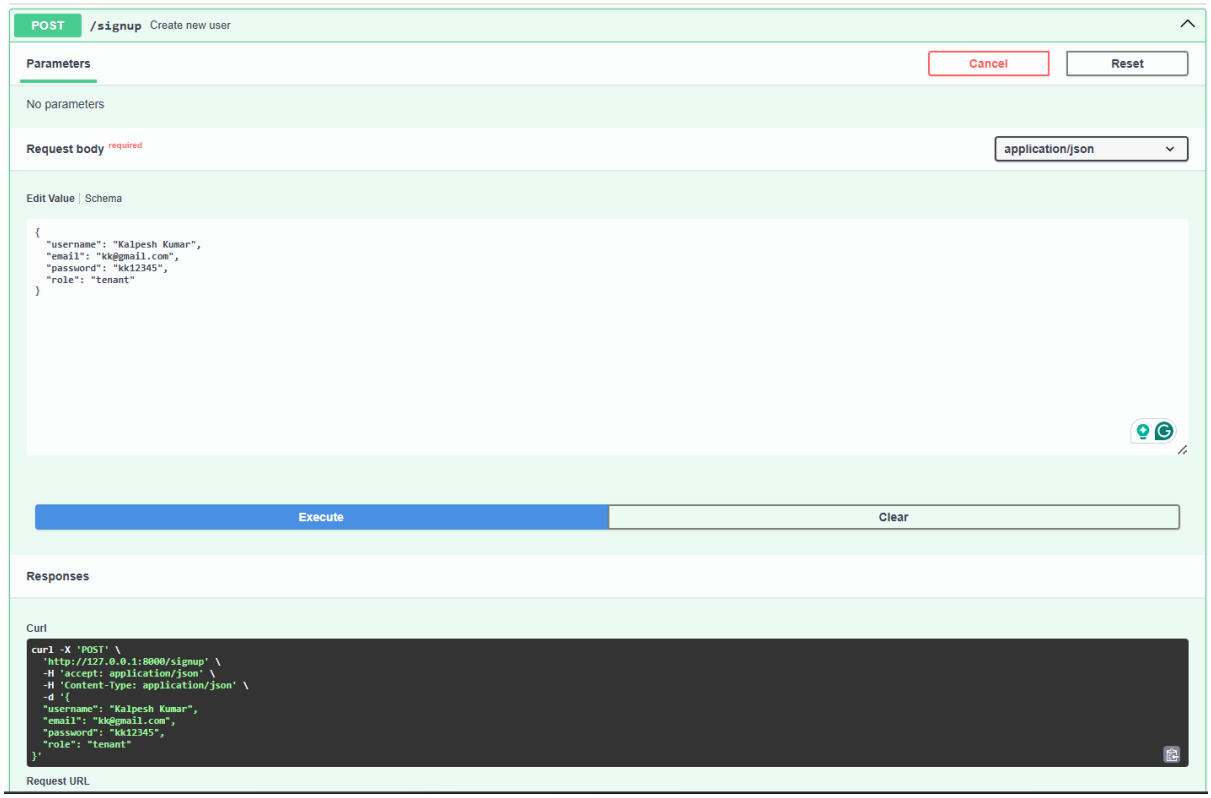


4.1 Request Flow and Business Rules

- **Authentication Check**: Endpoint paths that require authentication use the *get_current_user* dependency, which decodes the JWT token from the HTTP Bearer header, validates it against the signing key, and retrieves the matching user record from the database.
- **Role Restriction**: Landlord endpoints (e.g. creating/updating properties) are protected by *check_landlord*, which returns a 403 Forbidden error if a tenant tries to list a house.
- **Lease Automation**: When a tenant successfully leases a property (`POST /leases`), the system automatically registers the lease in the database and updates the property's `is_available` column to `False`. When a lease is terminated (`POST /leases/{id}/terminate`), the property availability is set back to `True`.

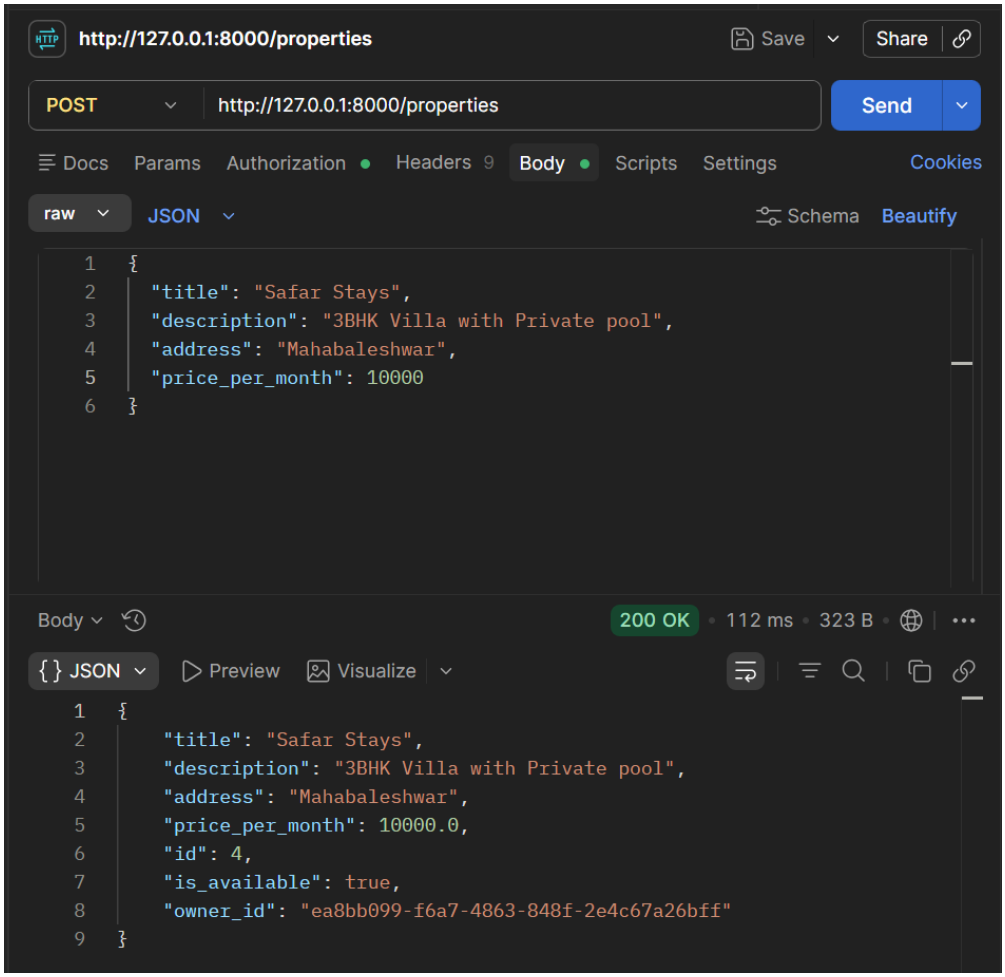
5. Postman Integration & Testing Guide

5.1 Sign Up (`POST /signup`)



5.2 Login & Token Retrieval (`POST /login`)

5.3 Calling Authenticated Routes (e.g., `POST /properties`)



5.4 Calling Authenticated Routes (e.g., `POST /properties`)